NexusDesk

Autonomous Multi-Agent IT Helpdesk with ServiceNow Integration

Phase 1: Requirements, Scope and Architecture Design

Design Document

Prepared by Nijitha Mol M

Contents: user stories, ticket categories and priority rules, architecture diagram, agent workflow diagram, action policy and thresholds, project structure, non-goals

1. Purpose and Key Design Decisions

This document fixes what NexusDesk will and will not do, and how its parts fit together, before any code is written. Later phases should follow it. If a later phase needs a change, update this document first so it stays the single source of truth for prompts.

Decisions made in Phase 1

Topic	Decision	Reason
Backend	One Python FastAPI service hosts the API, auth, guardrails, agents, RAG and workers. The optional Node.js gateway from the stack list is dropped.	Fewer moving parts and less deployment work. Agent code is Python anyway.
Frontend	React + Tailwind. One app with two role-based areas: Employee Portal and Admin Dashboard.	Reuses existing MERN skills; one deployment.
Ticket system	ServiceNow PDI is the system of record for incidents and knowledge. MongoDB stores only app data (users, chats, approvals, audit log, graph state).	Avoids duplicating ticket data; keeps ServiceNow integration meaningful.
Vector store	ChromaDB, local, persisted to disk.	Free, simple, enough for a few hundred documents.
Agent roles	Five roles: Supervisor, Triage, Resolver, Action, Reporter. Supervisor is routing logic in the graph (code plus a small LLM call only when needed), not a free-roaming agent.	Deterministic routing is safer and cheaper than letting an LLM decide every step.
Autonomy	Read and low-risk write operations are automatic. Resolving high-priority tickets and all sensitive requests need human approval.	Matches industry practice and the human-in-the-loop requirement.
Sensitive ops	Password reset and access grants are mocked. The system records the approved decision and writes it to the ticket, but changes no real account.	No real identity system exists; avoids unsafe behaviour.
Language	English only in v1.	Keeps evaluation manageable. Tamil is a future enhancement.
Data	Only synthetic data. No real employee data.	Privacy and safety.

2. Actors and User Stories

Actors: **Employee** (raises issues), **Admin** (approves actions, views reports), **Support Agent** (human in ServiceNow who receives escalated tickets), and the **System** (NexusDesk agents and workers). Priority: **M** = must have, **S** = should have.

ID	Story	Acceptance criteria	Pri
US-01	As an Employee, I want to describe my issue in plain English so that I do not need to fill a form.	Free-text message accepted; a ServiceNow ticket number is shown within the reply; invalid or empty input gets a clear message.	M
US-02	As the System, I want to triage the message so the ticket has the right category, urgency and impact.	Category is one of the supported list; urgency and impact are valid ServiceNow values; short description and description are filled; confidence is stored.	M
US-03	As the System, I want to ask a clarifying question when details are missing, so triage is reliable.	At most 2 clarification rounds; after that the ticket is escalated with the context gathered.	M
US-04	As an Employee, I want a step-by-step solution with sources so I can fix the issue myself.	Answer is based only on retrieved content; cites KB numbers; says clearly when nothing relevant is found.	M
US-05	As an Employee, I want to say whether the fix worked so the ticket is closed or progressed correctly.	Confirm leads to resolve flow; not fixed leads to one retry, then escalation.	M
US-06	As the System, I want to escalate tickets I cannot solve to the right human group, with full context.	Assignment group is set from the category mapping; work note summarises the issue, attempts and sources.	M
US-07	As an Admin, I want to approve or reject risky actions so that the agents cannot cause harm.	Pending approvals list shows proposal, reasoning and sources; decision is stored, written to the ticket work notes and resumes the workflow.	M
US-08	As an Employee, I want to see the status of my tickets so I know progress.	List and detail view reflect the current ServiceNow state.	M
US-09	As an Admin, I want a dashboard of ticket volume, categories, auto-resolution and escalation rates.	Metrics computed from stored data for a selectable date range.	S
US-10	As an Admin, I want a weekly trend report generated automatically.	Report lists top categories, recurring issues and suggested new KB articles.	S
US-11	As an Admin, I want to re-sync knowledge from ServiceNow so new articles are searchable.	Manual trigger and scheduled run; added, updated and removed articles are reflected in the vector store.	M
US-12	As a Support Agent, I want tickets created directly in ServiceNow to be picked up by NexusDesk.	New unassigned tickets are detected by polling and processed once only.	S
US-13	As an Employee, I want to rate the answer so quality can be tracked.	Thumbs up or down stored per answer and shown in analytics.	S
US-14	As the System, I want to reject malicious or off-topic input safely.	Prompt-injection attempts do not change agent behaviour; refusal is logged.	M

3. Ticket Categories and Priority Rules

Supported categories (v1)

Each category maps to a ServiceNow category and an escalation group. Confirm exact values in Phase 3.

NexusDesk category	Typical issues	ServiceNow category (to verify)	Escalation group (to create)
Network and VPN	VPN not connecting, Wi-Fi drops, slow internet	network	Network Support
Software and Access	App install, license, Jira or tool access, crashes	software	Software Support
Hardware	Laptop, monitor, keyboard, printer faults	hardware	Hardware Support
Email and Collaboration	Outlook, mail delivery, Teams or Slack problems	software (sub-category email)	Messaging Support
Account and Password	Locked account, password reset, MFA problems	inquiry	Service Desk
Other / Unknown	Anything unclear or outside the above	inquiry	Service Desk

Urgency and impact definitions

Value	Impact (how many affected)	Urgency (how badly work is blocked)
1 - High	Whole company or a department, or a critical service	Work fully blocked and no workaround
2 - Medium	A team or several users	Work slowed, a workaround exists
3 - Low	One user	Minor inconvenience, no effect on work

Priority matrix

ServiceNow calculates priority from urgency and impact; NexusDesk sets only those two. Default matrix (verify in the instance):

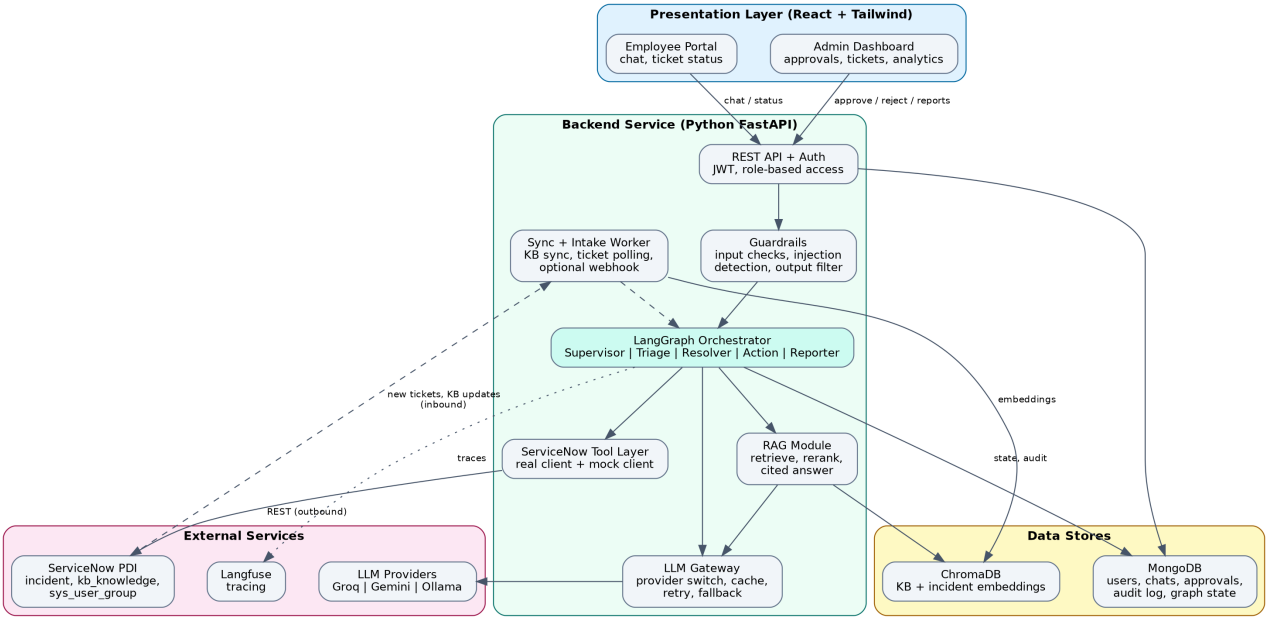
	Urgency 1 (High)	Urgency 2 (Medium)	Urgency 3 (Low)
Impact 1 (High)	P1 Critical	P2 High	P3 Moderate
Impact 2 (Medium)	P2 High	P3 Moderate	P4 Low
Impact 3 (Low)	P3 Moderate	P4 Low	P5 Planning

Special rules

- When unsure, triage chooses the **lower** impact (usually 3, one user) unless the user states that several people are affected. Overstating severity wastes human time.
- Keywords such as outage, everyone, all users, production down or security breach force a human review flag, regardless of the model's output.
- P1 tickets and security-related reports (phishing, suspected compromise, lost device) are never auto-resolved: they are created, then escalated to a human immediately.

4. System Architecture

The diagram shows the main components. Solid arrows are requests that start inside NexusDesk. Dashed arrows show inbound data from ServiceNow (new tickets and knowledge updates). The dotted arrow is tracing.



Component responsibilities

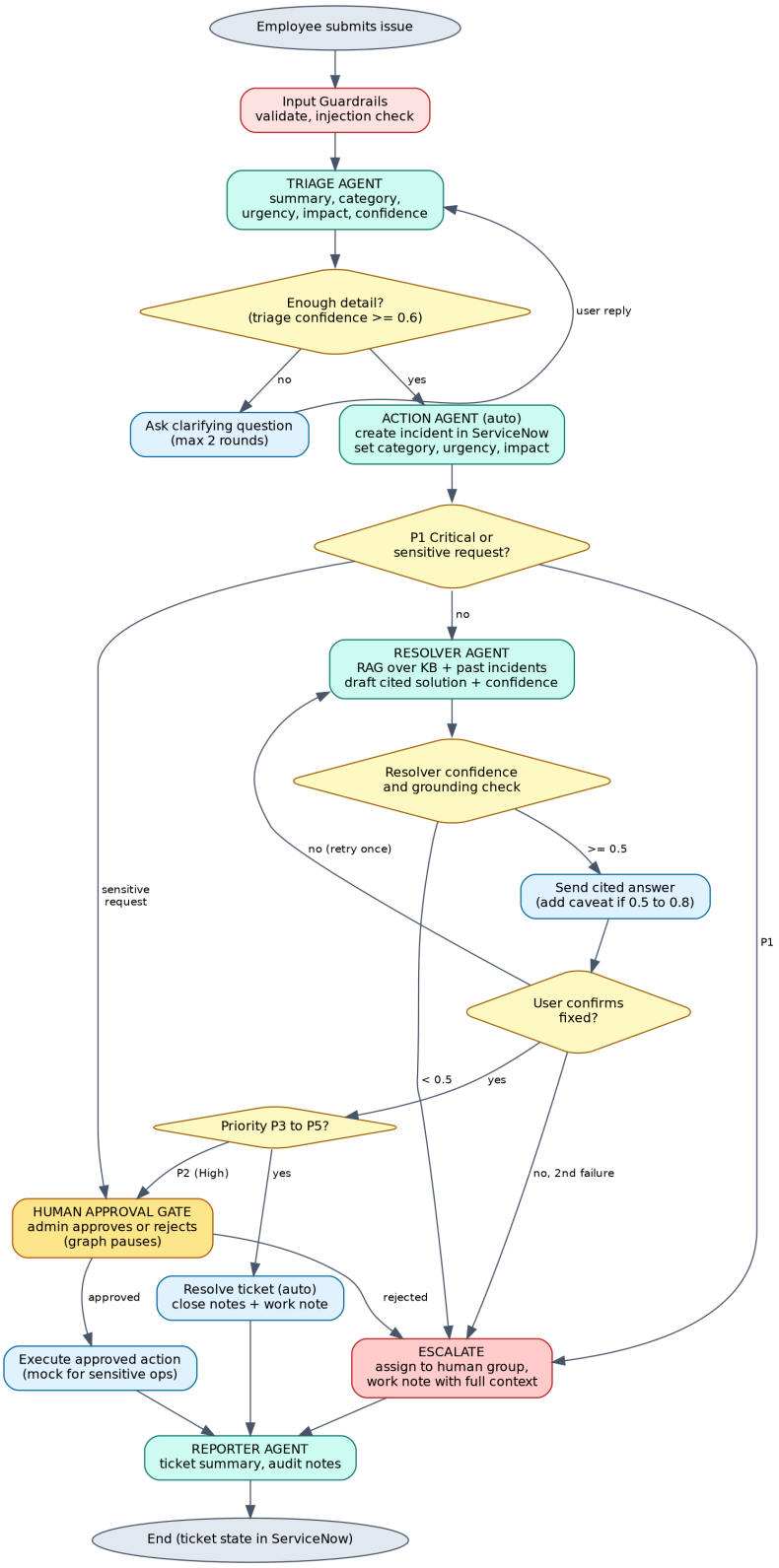
Component	Responsibility
Employee Portal / Admin Dashboard	User interfaces. Never call ServiceNow or the LLM directly; they use only the backend API.
REST API + Auth	Endpoints for chat, tickets, approvals, analytics and sync. JWT login and three roles: employee, support agent, admin.
Guardrails	Validates and limits input, detects prompt injection, and filters outputs before they reach the user or ServiceNow.
LangGraph Orchestrator	Holds the workflow graph, shared state, agents, routing and approval pauses. Saves state so paused tickets survive restarts.
RAG Module	Embeds queries, searches ChromaDB with category filters, optionally reranks, and returns passages with source metadata.
LLM Gateway	One interface to Groq, Gemini and Ollama with caching, retry, fallback, JSON-output validation and usage logging.
ServiceNow Tool Layer	Real and mock clients behind one interface. All ServiceNow access from agents goes through this layer as named tools.
Sync + Intake Worker	Pulls knowledge articles and resolved incidents into ChromaDB; polls ServiceNow for new tickets; optional webhook receiver.
MongoDB	Users, conversations, approvals, audit log, feedback and workflow checkpoints.
ChromaDB	Vector index for knowledge articles and past resolved incidents, with metadata (KB number, category, source type, date).
Langfuse	Traces every agent step, tool call, retrieval and LLM call.

Data flows

- **Outbound (NexusDesk to ServiceNow):** create incident, update fields, add work notes and comments, resolve, reassign, read knowledge articles.
- **Inbound (ServiceNow to NexusDesk):** new tickets created in ServiceNow, and changed or new knowledge articles. Polling first; webhook is optional.
- **Employee request path:** Portal, API, Guardrails, Orchestrator, (RAG, LLM Gateway, ServiceNow Tool Layer), response back to Portal.
- **Approval path:** Orchestrator pauses and stores a proposal; Admin Dashboard shows it; the decision goes through the API and the graph resumes.

5. Agent Workflow

Green boxes are agents, yellow diamonds are routing decisions, the amber box is the human approval gate and the red box is escalation to a human support group. Every path ends in the Reporter step so that each ticket has a summary and audit trail.



Shared workflow state

All agents read from and write to one state object stored with the workflow checkpoint. Agents must not keep hidden memory elsewhere.

Field	Meaning
conversation	Messages exchanged with the employee
user	Employee id, name, role
ticket	ServiceNow sys_id, number, state, assignment group
triage	Summary, category, urgency, impact, flags, triage confidence
clarify_rounds	Number of clarifying questions asked (max 2)
retrieved	Passages retrieved with source (KB number), score and category
proposed_answer	Drafted solution text with citations and resolver confidence
grounded	Result of the check that every claim is supported by retrieved text
attempts	Number of resolution attempts (max 2)
proposed_action	Action name, parameters, risk level, reason
approval	Status (not needed, pending, approved, rejected), decided by, time, comment
audit	Ordered list of decisions and events written to MongoDB and ServiceNow work notes
outcome	Auto-resolved, resolved after approval, escalated or abandoned

Agent inputs and outputs

Agent	Input	Output	Allowed tools
Triage	Conversation, user info	Structured JSON: summary, category, urgency, impact, flags, confidence, clarifying question if needed	None (no external calls)
Resolver	Triage result, conversation	Cited solution, confidence, list of sources, grounded flag	RAG search, incident search (read only)
Action	Decision from supervisor, ticket data	Executed tool result or an approval proposal	ServiceNow create, update, comment, work note, resolve, reassign
Reporter	Final state, ticket history	Ticket summary, resolution notes, weekly report text	Read-only data access
Supervisor	Current state	Next node name	None

Routing conditions

After step	Condition	Next step
Input guardrails	Input blocked	Refuse politely, log, end
Triage	Confidence below 0.6 and clarify rounds below 2	Ask clarifying question
Triage	Confidence below 0.6 and 2 rounds used	Create ticket as Other, then escalate
Ticket created	Priority P1, or a security keyword, or flagged for review	Escalate

After step	Condition	Next step
Ticket created	Request for access change or password reset	Approval gate (mocked execution)
Resolver	Confidence below 0.5, or answer not grounded, or nothing retrieved	Escalate
Resolver	Confidence 0.5 to 0.8	Send answer with caveat and offer human help
Resolver	Confidence 0.8 or above and grounded	Send answer
User feedback	Fixed, priority P3 to P5	Auto-resolve
User feedback	Fixed, priority P2	Approval gate, then resolve
User feedback	Not fixed, attempts below 2	Resolver retry with the new information
User feedback	Not fixed, 2 attempts used	Escalate
Approval gate	Approved	Execute action
Approval gate	Rejected, or no decision within 24 hours	Escalate to human group
Any node	Step limit (for example 12 steps) or tool error after retries	Escalate with error note

6. Action Policy and Confidence Thresholds

Every action is classified by risk. The Action Agent may only execute what the policy allows, and the policy is enforced in code, not only in the prompt.

Action	Risk	Policy
Search knowledge base and incidents	Low	Automatic
Create incident	Low	Automatic
Set category, urgency, impact	Low	Automatic
Add work note (internal)	Low	Automatic; every decision is logged this way
Add comment (visible to employee)	Low	Automatic, after output filter
Reassign to support group (escalate)	Low	Automatic
Resolve ticket, priority P3 to P5	Medium	Automatic only after employee confirms the fix
Resolve ticket, priority P2	High	Admin approval required
Access grant or password reset	High	Admin approval required; execution is mocked
Close or resolve P1 ticket	Critical	Never automatic; human only
Delete records, change user roles, touch tables other than incident and knowledge	Forbidden	Not exposed as tools

Initial confidence thresholds

Measure	Threshold	Effect
Triage confidence	Below 0.6	Ask clarifying question
Resolver confidence	Below 0.5	Escalate
Resolver confidence	0.5 to 0.8	Answer with caveat
Resolver confidence	0.8 or above	Answer normally
Top retrieval similarity	Below a tuned cut-off	Treat as nothing relevant found
Clarification rounds	Maximum 2	Then escalate
Resolution attempts	Maximum 2	Then escalate
Approval timeout	24 hours	Escalate to human group

Important: these numbers are starting values, not facts. A model's own confidence score is often poorly calibrated, so the final confidence should combine the model's score, the retrieval similarity and the grounding check. The thresholds will be tuned in Phase 15 using the labelled test tickets, and the final values should be recorded here.

7. Project Structure and Naming Conventions

Repository name: **nexusdesk** (lower case, hyphen-free). Planned layout:

```
nexusdesk/
  backend/
    app/
      api/            routes, request and response schemas
      core/           config, security, logging
      agents/         triage, resolver, action, reporter, supervisor
      graph/          state definition, graph builder, routing rules
      rag/            ingestion, chunking, retriever, answer builder
      llm/            gateway and provider adapters
      servicenow/     client interface, real client, mock client, tools
      guardrails/     input checks, output filters, policy rules
      workers/        KB sync, ticket intake, scheduled reports
      db/             MongoDB access and models
    tests/            unit, integration, edge-case tests
  frontend/
    src/              pages, components, API client, auth
  data/
    synthetic/        generated KB articles and incidents
    eval/             labelled test tickets (never indexed)
  evaluation/         scripts and result reports
  docs/               design document, diagrams, demo notes
  docker/             Dockerfiles, compose file
  .github/workflows/  CI pipelines
  .env.example  README.md
```

Conventions

- Python: snake_case for files, functions and variables; PascalCase for classes. Type hints and short docstrings on public functions.
- React: PascalCase component files, camelCase variables, one API client module.
- API: plural nouns, for example /tickets and /approvals, versioned under /api/v1.
- Git: branch per phase (for example phase-4-servicenow-client); commit messages in the form feat:, fix:, docs:, test:, chore:.
- Environment variables are upper case with the prefix of their service, for example SN_INSTANCE and LLM_PROVIDER. Secrets never enter Git.
- Every agent output must be a validated structured object, not free text, wherever another component consumes it.
- Prompts are stored as separate versioned files, not hard-coded inside functions.

8. Non-Goals (Scope Control)

- No real password resets, account changes or access grants.
- No other ITSM modules (change, problem, asset management) and no ServiceNow custom applications.
- No real employee or company data, and no storage of personal data beyond demo accounts.
- No voice, mobile app or multilingual support in v1.
- No model training or fine-tuning; only prompting and retrieval.
- No paid services. If a free tier disappears, the provider switch is used instead of paying.
- No multi-tenant mode, enterprise single sign-on, or Slack and email intake in v1.
- The system is a portfolio-grade demonstration and makes no production-readiness claims.

9. Risks and Open Questions

Risk or question	Mitigation or next step
ServiceNow PDI may sleep or be reclaimed	Mock client from the start; log in regularly; keep seed scripts to rebuild the instance.
Free LLM rate limits and model changes	Provider switch, caching, retries and fallback; keep prompts model-agnostic.
Free-tier hosting cold starts	Loading message in the UI; demo mode with the mock client.
Hallucinated solutions	Grounding check, mandatory citations, escalation when nothing is found.
Synthetic data may be too easy, inflating evaluation results	Write part of the test tickets by hand with different wording from the knowledge base; report this limitation.
Exact ServiceNow field values may differ from this document	Confirm in Phase 3 and update Sections 3 and 6.
Open: Admin and Support Agent roles	Decide in Phase 11 whether they are separate login roles or one.

10. Phase 1 Completion Checklist

- User stories reviewed and priorities agreed (Section 2).
- Categories, escalation groups and priority rules agreed (Section 3).
- Architecture diagram agreed (Section 4).
- Workflow diagram, state, routing rules agreed (Section 5).
- Action policy and starting thresholds agreed (Section 6).
- Folder structure and conventions agreed (Section 7).
- Non-goals confirmed (Section 8).
- Next: Phase 2, environment and repository setup using the structure in Section 7.